Task & Document Management System

Highly Available Three-Tier Web Application on Amazon Web Services

A complete cloud infrastructure project: VPC, EC2, ALB, Auto Scaling, RDS, S3, IAM, CloudWatch, SNS

Prepared by: **Prashant Suryawanshi**

Project Specifications

Item	Details
Technology Stack	React, NGINX, Node.js (Express), MySQL, AWS
Cloud Platform	Amazon Web Services (AWS)
Region Used	us-east-1
Document Type	Step-by-Step Technical Documentation

Table of Contents

- 1. Project Overview & Architecture
- 2. Technology Stack
- 3. Architecture Flow Diagram
- 4. Step 1 - VPC & Networking Setup
- 5. Step 2 - Security Groups Configuration
- 6. Step 3 - IAM Role Creation
- 7. Step 4 - S3 Bucket for Document Storage
- 8. Step 5 - RDS MySQL Database Setup
- 9. Step 6 - Target Groups & Application Load Balancers
- 10. Step 7 - Launch Templates (Backend & Frontend)
- 11. Step 8 - Auto Scaling Groups
- 13. Step 9 - End-to-End Verification
- 14. Roles & Responsibilities

1. Project Overview & Architecture

The Task & Document Management System is a cloud-hosted web application where users can create and track tasks (with priority, status and due date) and upload, download and delete related documents. It has a React frontend and a Node.js (Express) REST API backend, and uses MySQL for data storage.

The project is deployed on AWS as a **three-tier architecture** inside a custom VPC spread across three Availability Zones:

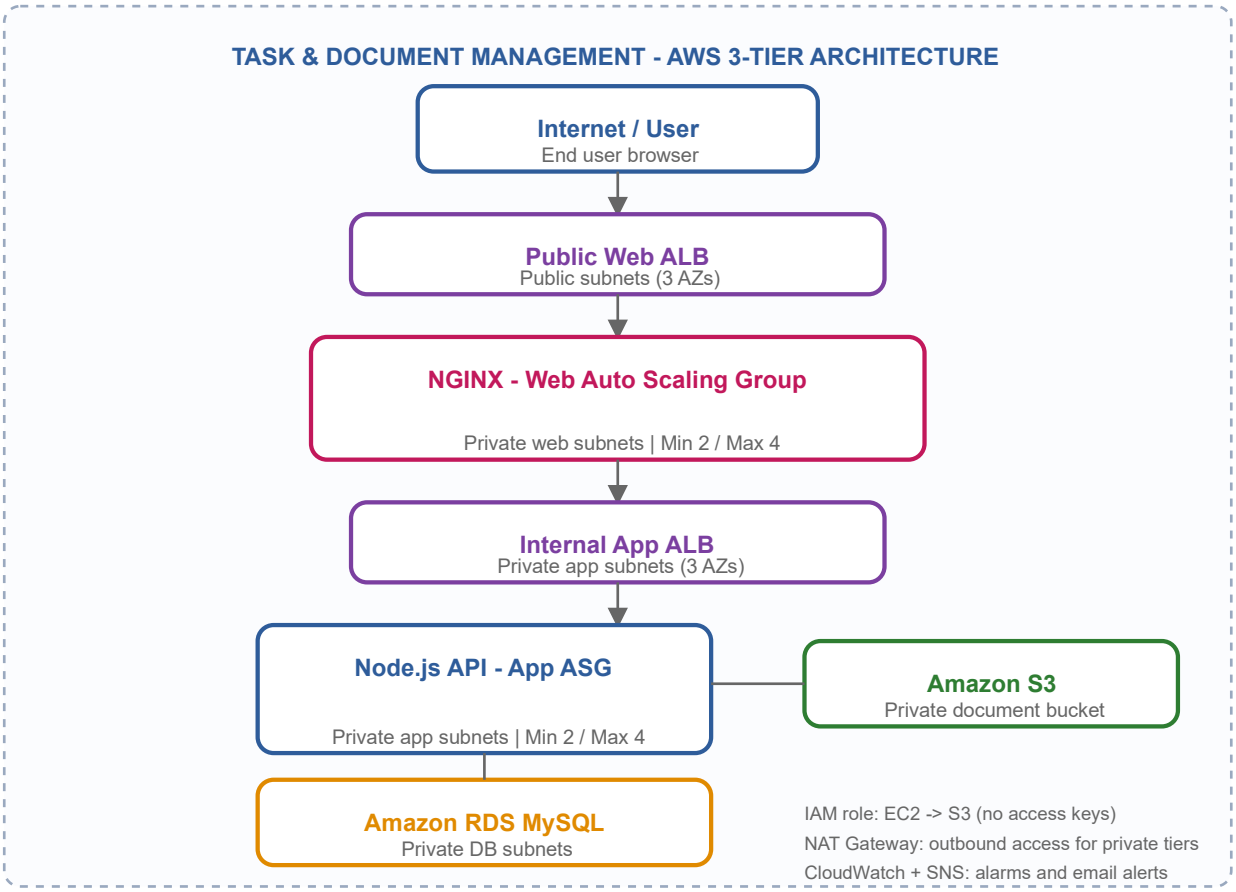
- **Tier 1, Presentation:** a public Application Load Balancer sends traffic to NGINX servers in an Auto Scaling Group. The NGINX servers sit in private subnets.
- **Tier 2, Application:** an internal Application Load Balancer sends traffic to Node.js servers in a second Auto Scaling Group, also in private subnets.
- **Tier 3, Data:** Amazon RDS MySQL in private database subnets stores task data and document metadata. A private Amazon S3 bucket stores the uploaded files.

No server is directly exposed to the internet. Only the Web ALB is public, and EC2 instances access S3 through an IAM role, so no access keys are stored in the application.

2. Technology Stack

Layer	Technology / Service	Purpose
Frontend	HTML, CSS, React (CDN)	User interface for tasks and documents
Web Server	NGINX	Serves the frontend and proxies /api requests to the backend
Backend	Node.js 20, Express	REST API, validation and file upload handling
Database	Amazon RDS MySQL	Stores tasks and document metadata
Object Storage	Amazon S3 (private)	Stores uploaded documents
Networking	VPC, Subnets, IGW, NAT Gateway, Route Tables	Isolated and secure cloud network
Compute	Amazon EC2 (Amazon Linux 2023)	Web and application servers
Load Balancing	Public ALB + Internal ALB	Traffic distribution for each tier
Scaling	Launch Templates + Auto Scaling Groups	Horizontal scaling and self-healing
Security	Security Groups, IAM Roles	Tier-to-tier access control, keyless S3 access
Monitoring	CloudWatch + SNS	Metrics, alarms and email alerts

3. Architecture Flow Diagram

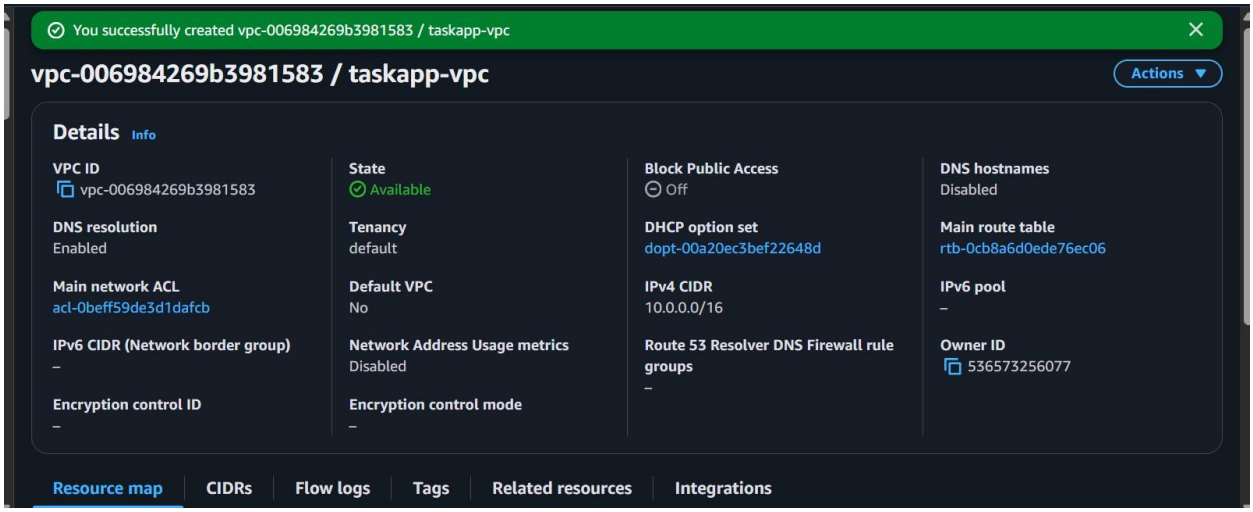


Request flow: Browser -> Web ALB -> NGINX (Web ASG) -> Internal App ALB -> Node.js (App ASG) -> RDS MySQL / S3

A user request reaches the public Web ALB, which forwards it to a healthy NGINX instance. NGINX serves the React frontend directly. Any request starting with /api is proxied to the internal App ALB, which forwards it to a Node.js instance. The API reads and writes task data in RDS and stores or retrieves uploaded documents in S3 using its IAM role.

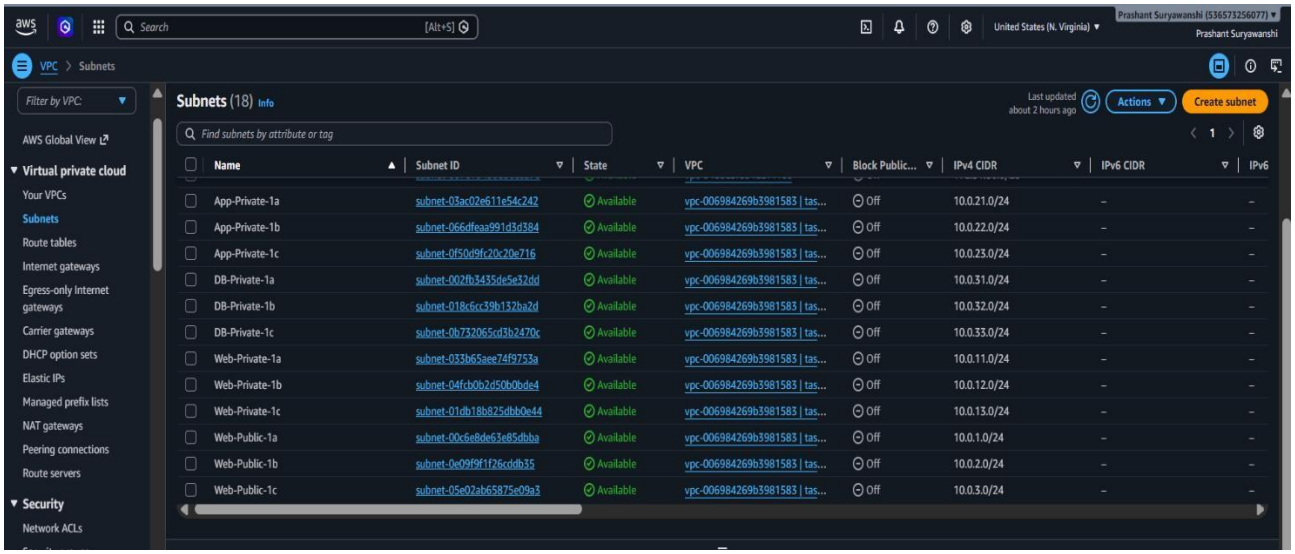
4. Step 1 - VPC & Networking Setup

1.1 Create VPC: A custom VPC **taskapp-vpc** was created with CIDR block 10.0.0.0/16.

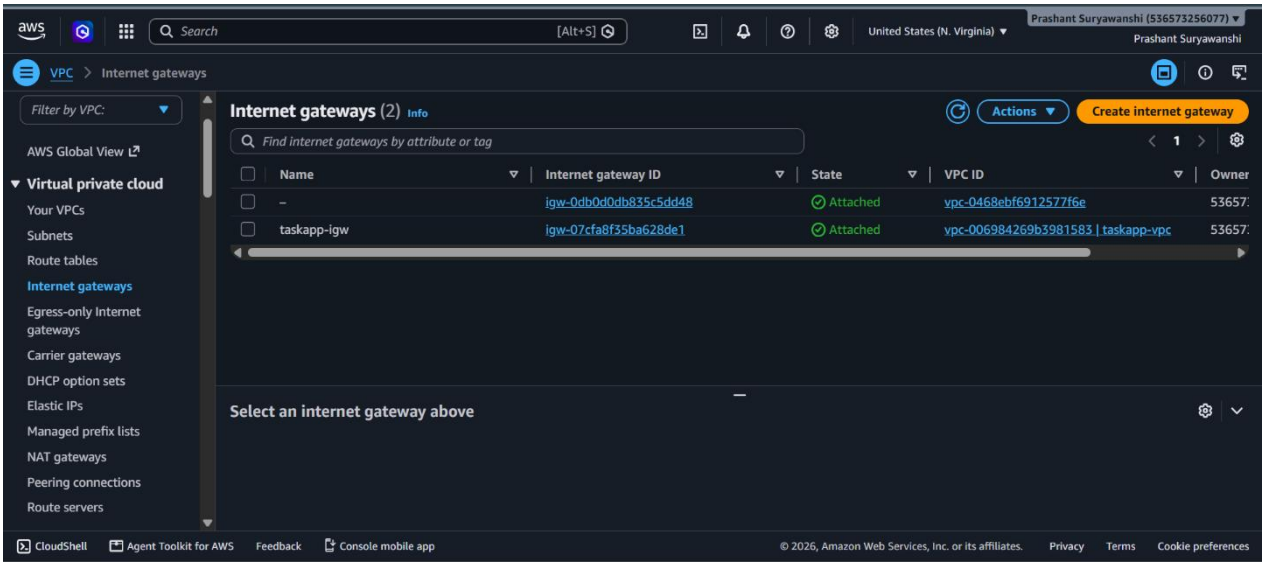


1.2 Create 12 Subnets across 3 Availability Zones:

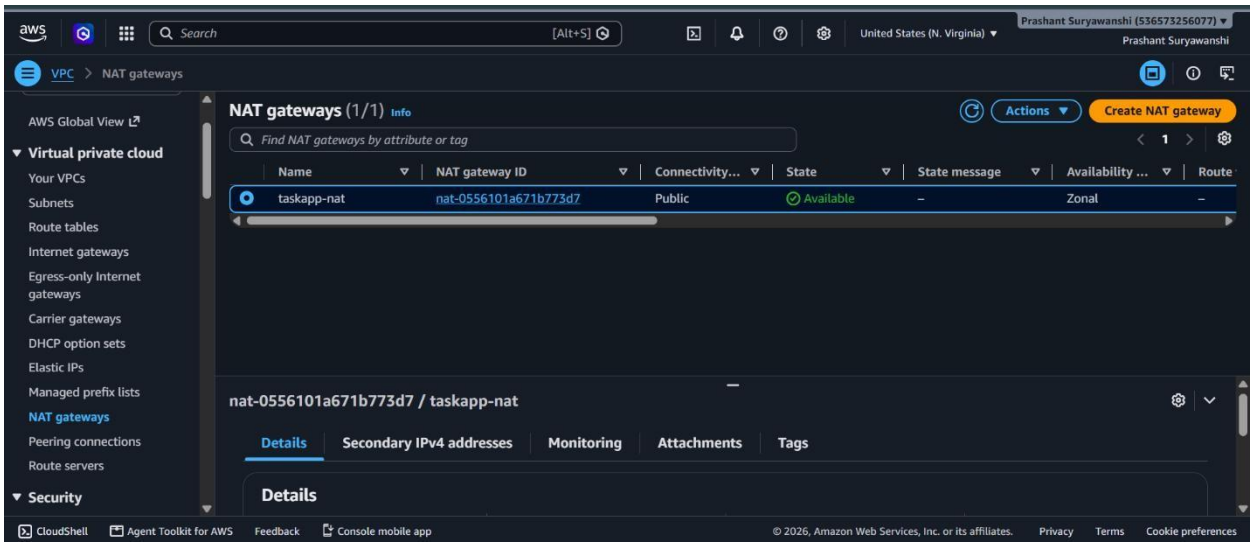
Subnet Group	AZ-a	AZ-b	AZ-c
Public	10.0.1.0/24	10.0.2.0/24	10.0.3.0/24
Private Web	10.0.11.0/24	10.0.12.0/24	10.0.13.0/24
Private App	10.0.21.0/24	10.0.22.0/24	10.0.23.0/24
Private DB	10.0.31.0/24	10.0.32.0/24	10.0.33.0/24



1.3 Internet Gateway: An IGW was created and attached to the VPC so public subnets can reach the internet.

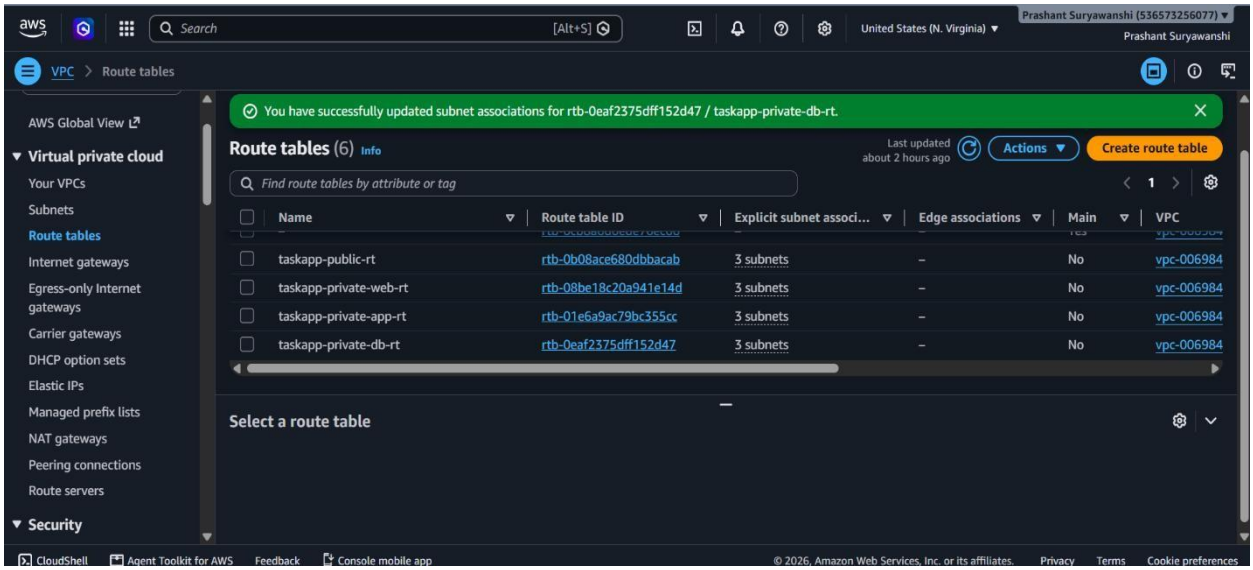


1.4 NAT Gateway: A NAT Gateway was deployed in the first public subnet so private web and app servers can download updates and packages without being exposed to the internet.



1.5 Route Tables:

- Public route table: 0.0.0.0/0 to Internet Gateway
- Private Web and App route tables: 0.0.0.0/0 to NAT Gateway
- DB route table: local VPC route only (the database needs no internet access)



5. Step 2 - Security Groups Configuration

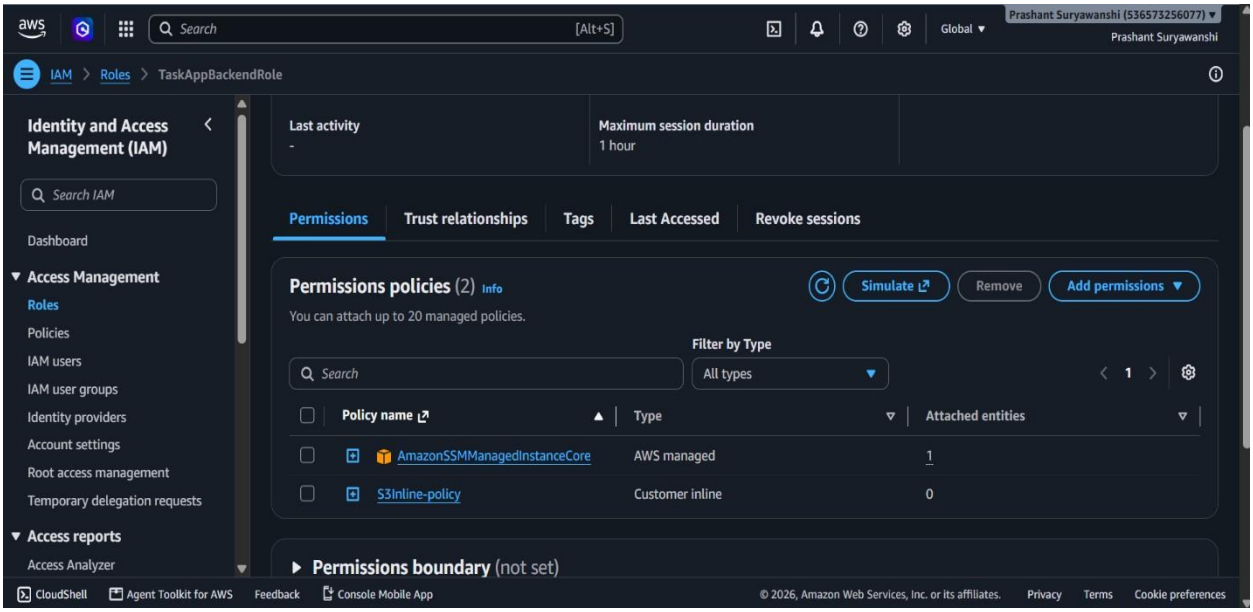
Five security groups were created so that each tier only accepts traffic from the tier directly in front of it:

Security Group	Inbound Rule	Source
sg-web-alb	HTTP 80	0.0.0.0/0
sg-web-server	HTTP 80	sg-web-alb
sg-app-alb	HTTP 80	sg-web-server
sg-app-server	TCP 3000	sg-app-alb
sg-rds	MySQL 3306	sg-app-server only

6. Step 3 - IAM Role Creation

An IAM role **TaskAppBackendRole** was created for EC2 so that backend servers can access S3 without hard-coded access keys.

- **Trusted entity:** AWS service, EC2
- **Managed policy attached:** AmazonSSMManagedInstanceCore (allows access through Session Manager without SSH)
- **Inline policy:** allows s3:GetObject, s3:PutObject, s3:DeleteObject and s3:ListBucket on the project bucket only (least privilege)



7. Step 4 - S3 Bucket for Document Storage

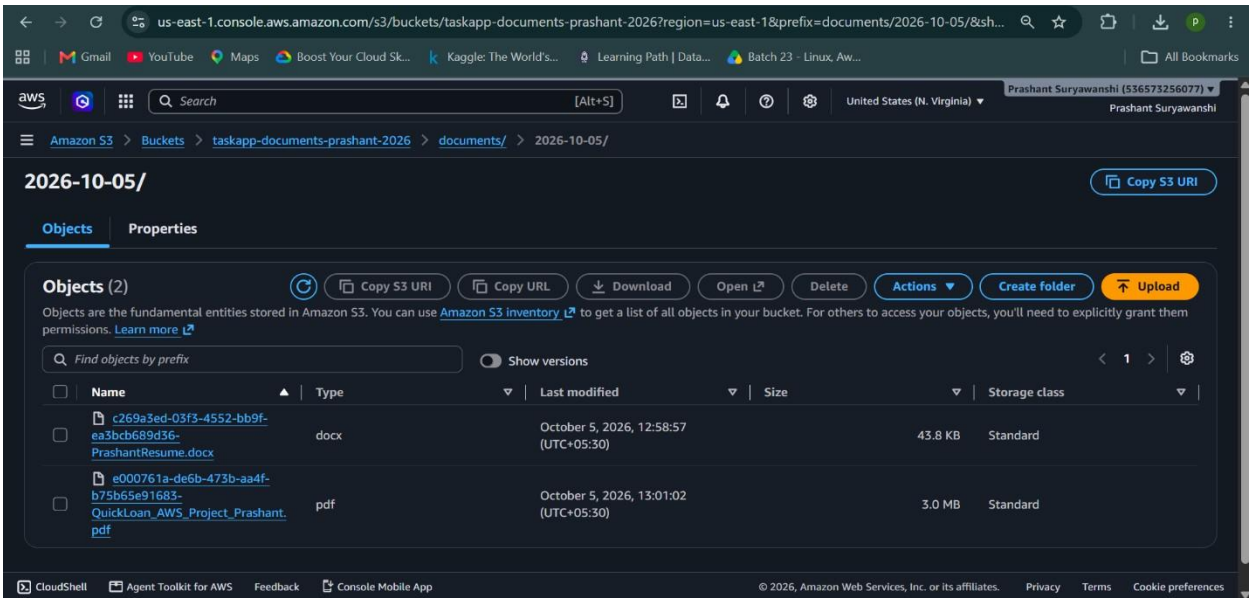
A private S3 bucket **[your-bucket-name]** was created to store uploaded documents.

- Block all public access: **ON**
- Object Ownership: Bucket owner enforced
- Versioning: **Enabled**
- Default encryption: SSE-S3

Download: the API reads the s3_key from RDS and streams the file from S3 to the browser.

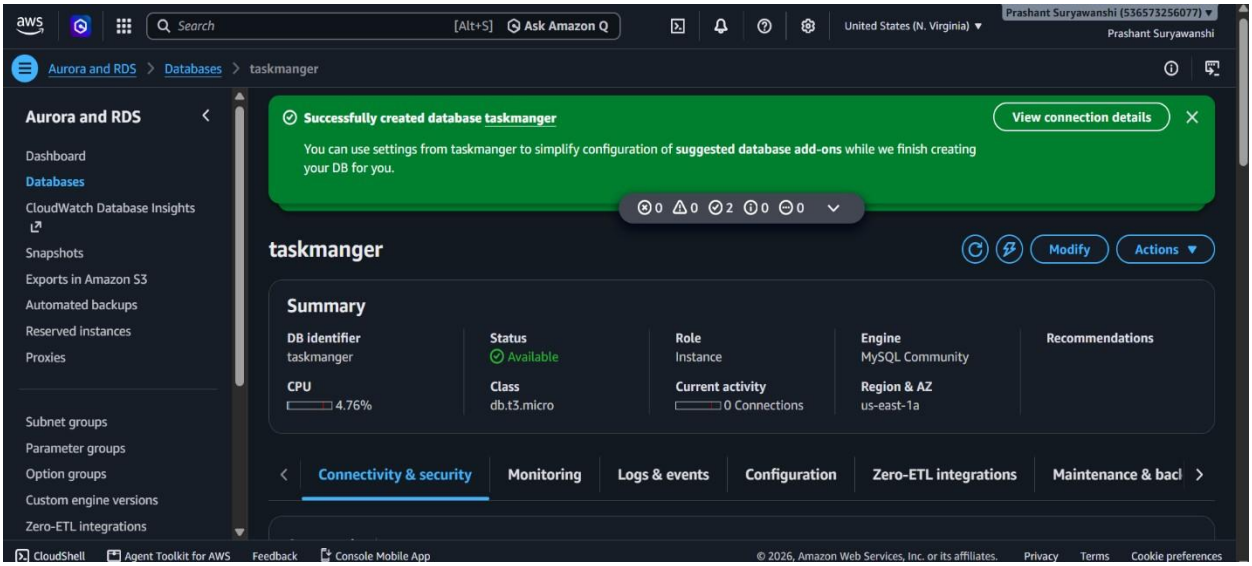
Delete: the API removes the row from RDS and deletes the object from S3. Because versioning is enabled, S3 adds a **delete marker** and the object disappears from the normal list. Turning on **Show versions** displays the earlier version, which can be restored if deleted by mistake

Document files are stored in S3, while RDS stores only their metadata and S3 key.



8. Step 5 - RDS MySQL Database Setup

- **DB subnet group** created using the three private DB subnets
- **Engine:** MySQL 8.x, **DB name:** task_manager
- **Public access:** No, **Security group:** sg-rds
- **Automated backups:** Enabled



After the instance became available, the schema was created from a backend EC2 instance: `mysql -h <RDS-ENDPOINT> -u <username> -p < /opt/taskapp/database/database_setup.sql`

This created the tasks and documents tables with sample data.

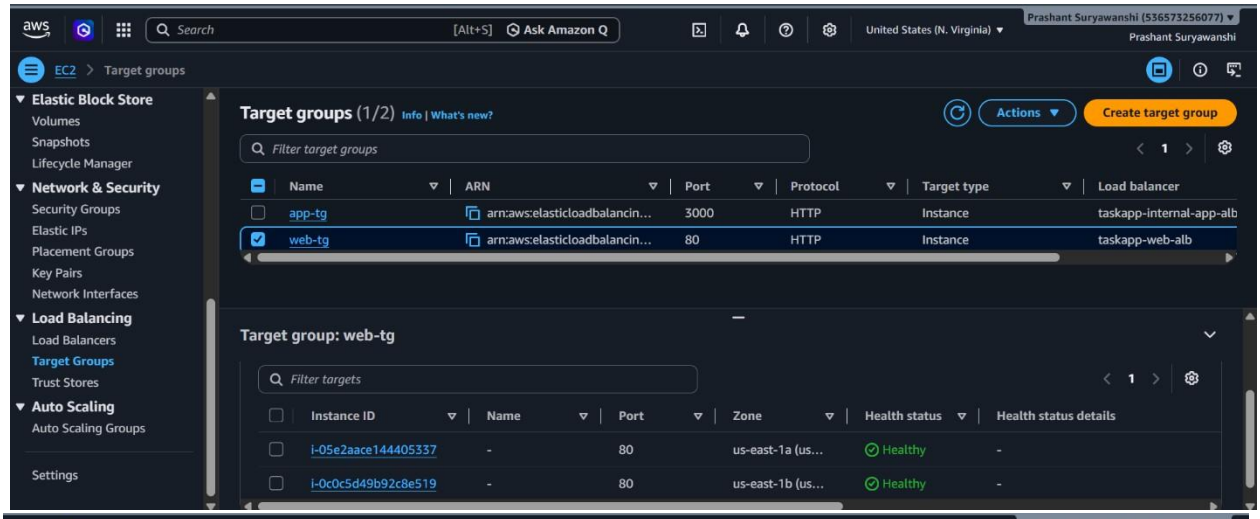
9. Step 6 - Target Groups & Application Load Balancers

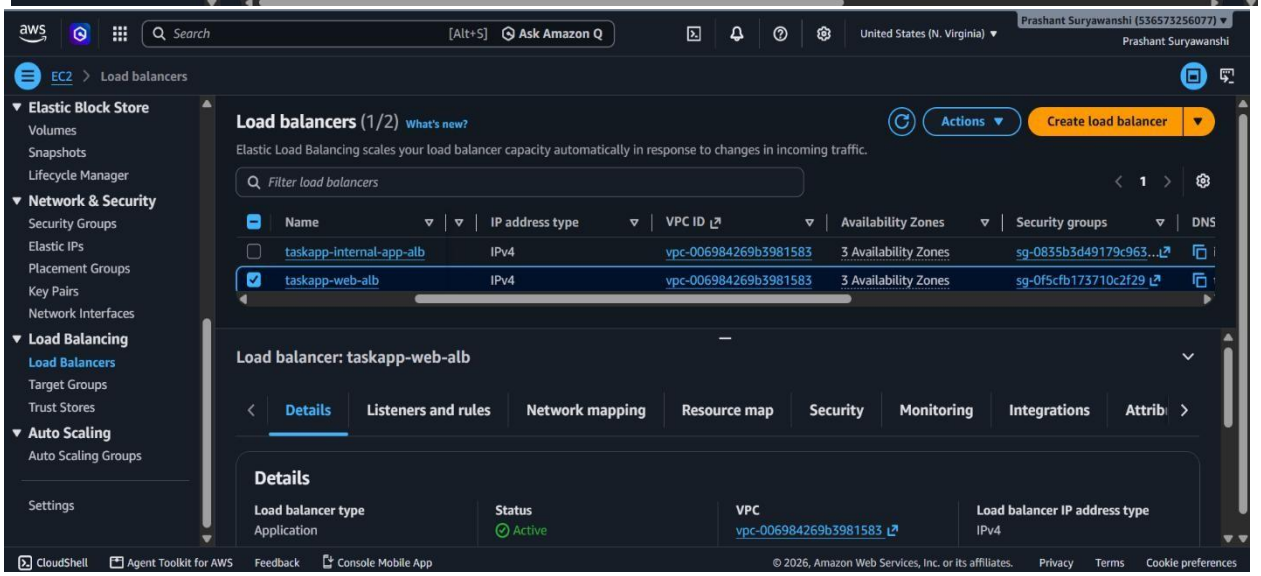
App tier (created first):

- Target Group **app-tg**: HTTP port 3000, health check path /api/health
- **Internal** ALB across the three private App subnets, using sg-app-alb, forwarding port 80 to app-tg

Web tier:

- Target Group **web-tg**: HTTP port 80, health check path /healthz
- **Internet-facing** ALB across the three public subnets, using sg-web-alb, forwarding port 80 to web-tg





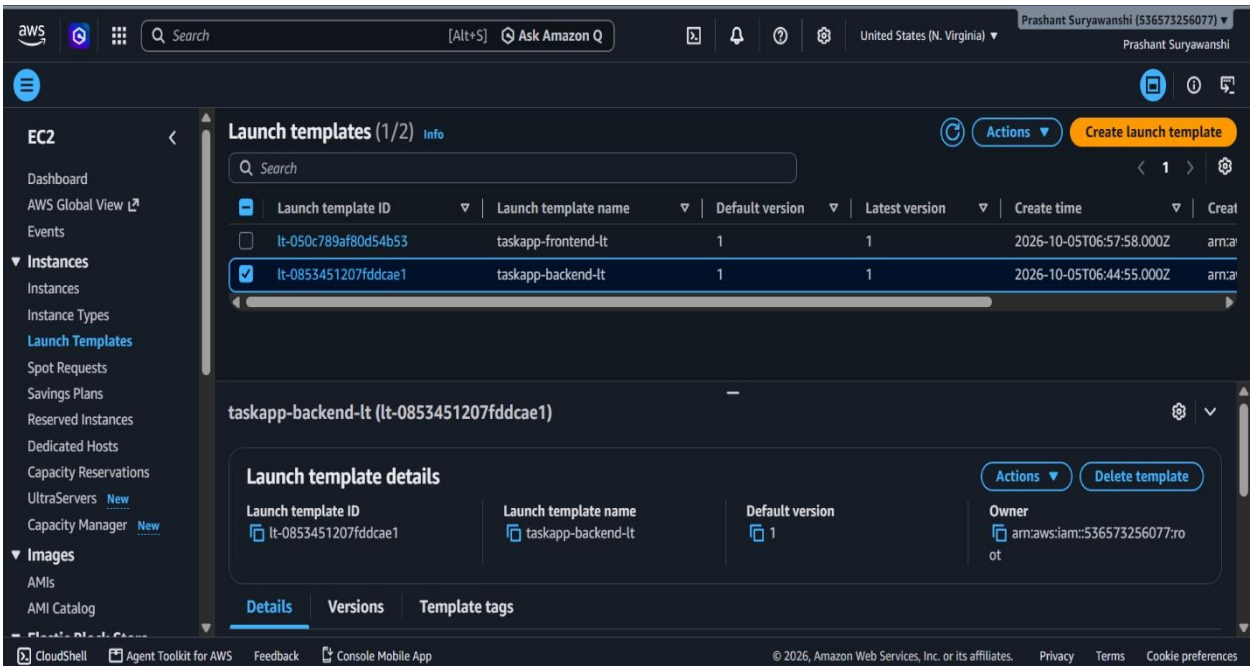
10. Step 7 - Launch Templates

Backend Launch Template: Amazon Linux 2023, instance profile TaskAppBackendRole, security group sg-app-server. The User Data script:

- Installs Node.js 20, npm, Git and the MariaDB client
- Clones the project and runs npm install --omit=dev
- Creates the .env file with the RDS endpoint and S3 bucket name
- Creates and starts a taskapi systemd service

Frontend Launch Template: Amazon Linux 2023, security group sg-web-server. The User Data script:

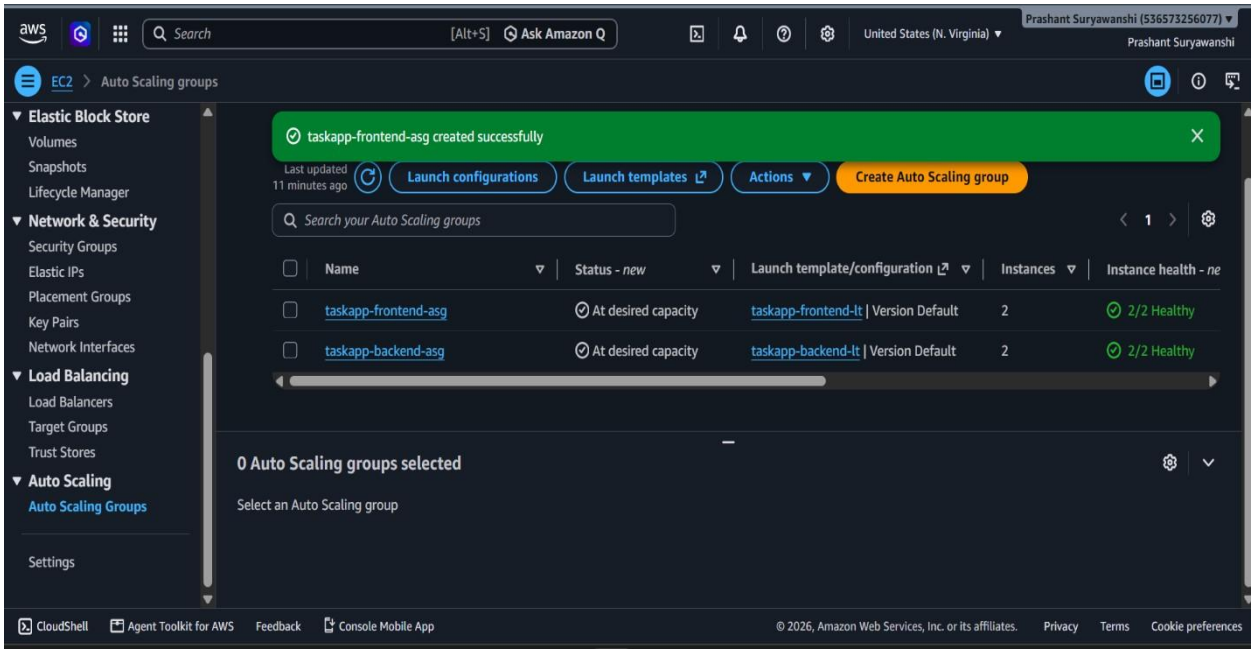
- Installs NGINX and Git
- Copies the frontend files to /usr/share/nginx/html/
- Generates the NGINX config with the **internal App ALB DNS name** as the /api/ proxy target
- Enables and starts NGINX



11. Step 8 - Auto Scaling Groups

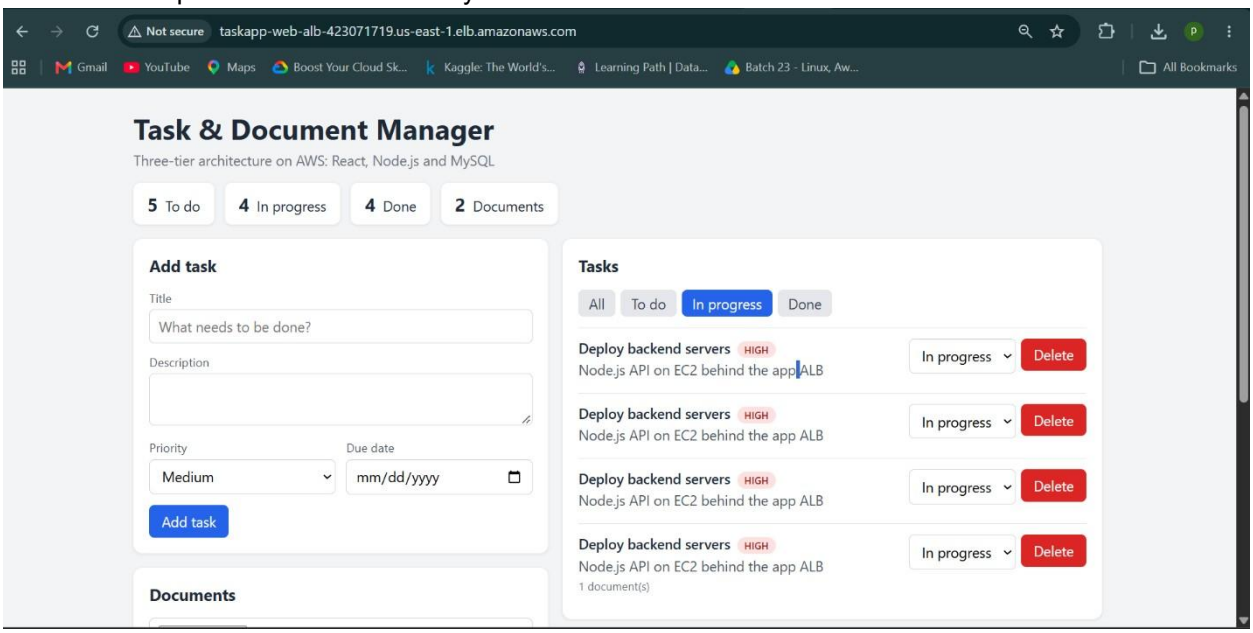
	App ASG	Web ASG
Subnets	3 private App subnets	3 private Web subnets
Target group	app-tg	web-tg
Min / Desired / Max	2 / 2 / 4	2 / 2 / 4
Scaling policy	Target tracking, ~50-60% CPU	Target tracking, ~50-60% CPU

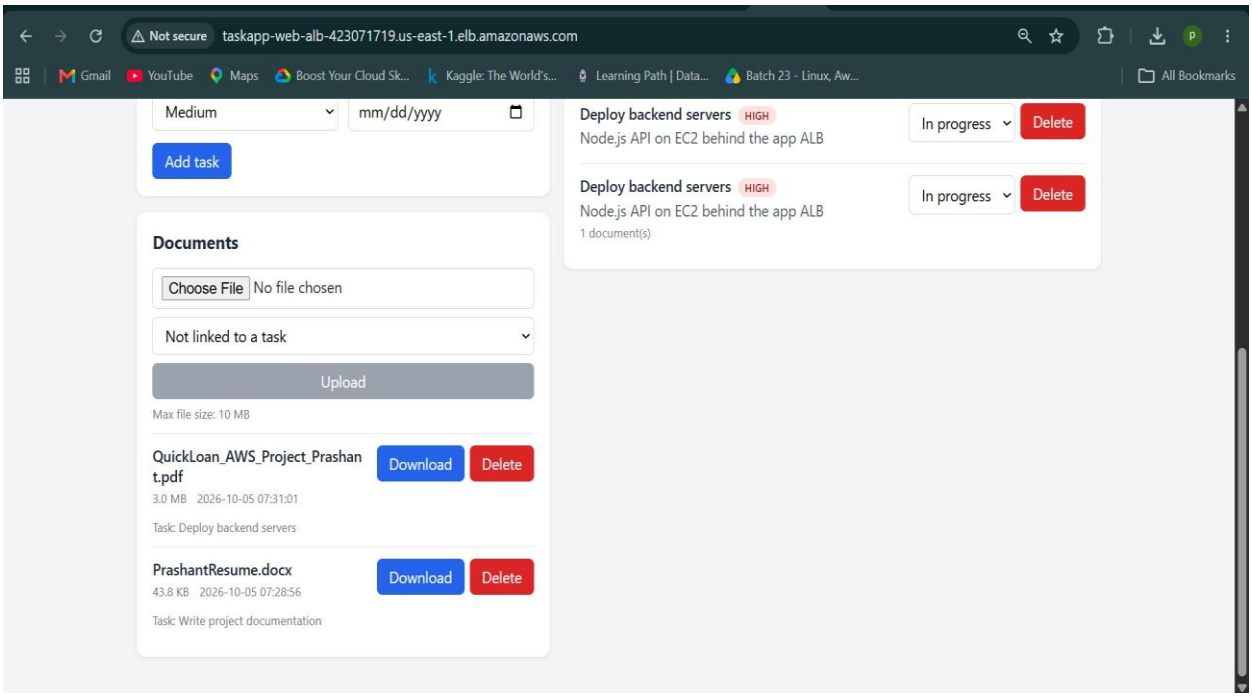
The App ASG was created first, and once its targets were healthy, the Web ASG was created using the App ALB DNS name.



12. Step 09 - End-to-End Verification

1. Opened the Web ALB DNS name in a browser and the Task & Document dashboard loaded.
2. Created a task, changed its status and verified it saved to RDS.
3. Uploaded a document and confirmed the file appeared in the S3 bucket.
4. Downloaded and deleted the document, and confirmed removal from S3.
5. **Auto Scaling test:** terminated one Web and one App instance, and the Auto Scaling Groups launched replacements automatically.





13. Roles & Responsibilities

#	Responsibility	Status
1	Designed the 3-AZ VPC with public, web, app and DB subnets, IGW, NAT Gateway and route tables	Completed
2	Configured chained Security Groups for each tier	Completed
3	Created IAM role with least-privilege S3 access for EC2	Completed
4	Configured private, versioned and encrypted S3 bucket for documents	Completed
5	Provisioned RDS MySQL in private subnets and created the schema	Completed
6	Implemented public and internal Application Load Balancers with target groups	Completed
7	Created Launch Templates with User Data and configured Auto Scaling Groups	Completed
8	Performed end-to-end testing and Auto Scaling failover testing	Completed